

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ХАРКІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ РАДІОЕЛЕКТРОНІКИ

Кафедра Програмної інженерії

Звіт з лабораторної роботи №3
з дисципліни: «Методи оптимізації та теорія ігор»
з теми: «Програмна реалізація методів підтримки колективного прийняття рішень»

Виконав:
ст. гр. ПЗПІ-23-8
Потурайко Н.І.

Перевірів:
асистент каф. ПІ
Демчук В. Г.

3 ПРОГРАМНА РЕАЛІЗАЦІЯ МЕТОДІВ ПІДТРИМКИ КОЛЕКТИВНОГО ПРИЙНЯТТЯ РІШЕНЬ

3.1 Мета роботи

Придбання навичок розробки та використання методів підтримки колективного прийняття рішень. Набуття вмінь формувати профіль голосування на основі індивідуальних ранжувань експертів та застосовувати математичні методи групового вибору для визначення найкращої альтернативи з урахуванням різних вагових коефіцієнтів учасників опитування. Додатковою метою є усвідомлення важливості переходу від суб'єктивних індивідуальних оцінок до об'єктивізованої групової моделі, що дозволяє мінімізувати ризики помилок при виборі технологічного стеку для програмних проектів.

3.2 Опис обраного методу прийняття рішень

Вибір єдиного правильного варіанту в групі часто супроводжується труднощами, оскільки індивідуальні вподобання експертів можуть кардинально відрізнятися. Класичні системи, такі як правило відносної більшості, часто призводять до парадоксів, коли переможцем стає альтернатива, яка є найменш бажаною для значної частини колективу. Для розв'язання задачі колективного вибору у даній роботі було застосовано більш досконалий алгоритм, а саме метод з урахуванням компетентності експертів.

Даний підхід є глибокою модифікацією класичного правила Борда. Звичайне правило Борда передбачає просте додавання балів за місця, але воно передбачає абсолютну рівноправність усіх виборців. У реальному житті, особливо у контексті розробки ігрового проекту, рівноправність є неефективною. Думка головного розробника або архітектора бази даних має набагато вагоміший вплив на фінальне технічне рішення, ніж думка молодшого тестувальника або дизайнера рівнів. Саме тому в алгоритм впроваджено систему зважування.

Математична модель розробленого методу базується на двох ключових компонентах. Першим компонентом є бали за здобуті місця. Оскільки досліджувана множина містить шість ігрових рушіїв, альтернатива на першому місці отримує п'ять балів, на другому отримує чотири бали, а на останньому шостому місці

отримує нуль балів. Формула розрахунку базових балів виглядає як загальна кількість альтернатив мінус поточне місце альтернативи у рейтингу експерта. Такий підхід заохочує рушії, які стабільно займають високі та середні позиції у більшості виборців.

Другим компонентом є вага самого експерта. У структурі бази даних кожен експерт має показник компетентності від одного до п'яти, де одиниця є найвищим можливим рангом управління. Вага для математичної формули обчислюється як константа шість мінус ранг експерта. Таким чином головний розробник отримує максимальну вагу п'ять, керівник проекту отримує вагу чотири, викладач має вагу три, геймдизайнер отримує вагу два, а інженер з тестування отримує найменшу вагу один.

Підсумковий бал кожної ігрової платформи є математичною сумою добутків балів за місце на вагу відповідного експерта по всьому профілю голосування. Переможцем беззаперечно визнається ігровий рушій з максимальною сумою зважених балів, що відображає справжній консенсус команди фахівців.

3.3 Алгоритм програмної реалізації

Програмна реалізація багатокритеріального методу була виконана на базі серверної платформи Node.js та сучасного клієнтського фреймворку Vue. Інформація для розрахунків не вводиться вручну, а автоматично завантажується з реляційної бази даних SQLite. У цій базі надійно зберігаються результати попередньої лабораторної роботи, де кожен експерт сформував свій строгий порядок рушіїв.

На стороні сервера створено два окремі програмні інтерфейси для обробки запитів. Перший інтерфейс відповідає виключно за формування профілю голосування. Він виконує складний реляційний запит до бази даних з багаторазовим об'єднанням таблиць результатів, експертів та альтернатив. Після отримання сирих даних сервер групує їх за ідентифікатором експерта та повертає у вигляді структурованого масиву. У цьому масиві кожен експерт має власний впорядкований перелік обраних рушіїв від першого до шостого місця.

Другий програмний інтерфейс виконує безпосередній математичний розрахунок за методом зважених балів. Базові дані групуються за ідентифікатором

альтернативи. Для кожного знайденого запису серверна логіка обчислює кількість балів за місце та поточну вагу експерта, після чого вони перемножуються і додаються до загального акумульованого рахунку рушія. Сервер також виконує важливу функцію генерації текстового рядка з детальним описом арифметичних дій. Це необхідно для візуалізації формули на стороні клієнта. Після завершення циклу обробки всіх записів масив сортується за спаданням підсумкового балу, виводячи лідера на першу позицію.

На стороні клієнта обидва запити виконуються паралельно за допомогою асинхронних функцій під час монтування компонента веб-сторінки. Отримані об'єкти даних прив'язуються до реактивних змінних. Завдяки реактивності фреймворку система автоматично генерує та наповнює дві таблиці користувацького інтерфейсу без необхідності перезавантаження сторінки.

3.4 Опис тестового прикладу (профіль голосування та результати)

Для перевірки працездатності розробленого програмного забезпечення було сформовано тестову групу з п'яти експертів. Кожному експерту було призначено відповідний рівень компетентності, який конвертується у ваговий коефіцієнт для розрахунків: Студент (вага п'ять), Керівник проекту (вага чотири), Викладач (вага три), Геймдизайнер (вага два) та QA Інженер (вага один).

Кожен учасник сформував власне строге ранжування шести ігрових рушіїв. Зведені дані індивідуальних вподобань утворюють профіль голосування, який наведено у таблиці нижче.

Таблиця 3.1 – Профіль голосування експертної групи

Місце	Студент (5)	Керівник (4)	Викладач (3)	Геймдизайнер (2)	QA Інженер (1)
1	Godot Engine	Unity	Unreal Engine 5	Defold	Unity
2	Unreal Engine 5	Unreal Engine 5	Unity	Godot Engine	Godot Engine
3	Unity	Godot Engine	Godot Engine	GameMaker	Unreal Engine 5
4	Defold	GameMaker	CryEngine	Unity	GameMaker
5	GameMaker	Defold	GameMaker	Unreal Engine 5	Defold
6	CryEngine	CryEngine	Defold	CryEngine	CryEngine

Бали за місця розподіляються наступним чином: перше місце дає п'ять балів, друге дає чотири бали, третє дає три бали, четверте дає два бали, п'яте дає один бал, шосте дає нуль балів.

Алгоритм виконує розрахунок для кожної альтернативи шляхом перемноження бала за місце на вагу експерта. Результати застосування методу групового вибору з урахуванням компетентності наведені у підсумковій таблиці.

Таблиця 3.2 – Результати колективного вибору з урахуванням компетентності

Місце	Ігровий рушій	Детальний розрахунок	Підсумковий бал
1	Godot Engine	$(5 \times 5) + (3 \times 4) + (3 \times 3) + (4 \times 2) + (4 \times 1)$	58
2	Unity	$(3 \times 5) + (5 \times 4) + (4 \times 3) + (2 \times 2) + (5 \times 1)$	56
3	Unreal Engine 5	$(4 \times 5) + (4 \times 4) + (5 \times 3) + (1 \times 2) + (3 \times 1)$	56
4	Defold	$(2 \times 5) + (1 \times 4) + (0 \times 3) + (5 \times 2) + (1 \times 1)$	25
5	GameMaker	$(1 \times 5) + (2 \times 4) + (1 \times 3) + (3 \times 2) + (2 \times 1)$	24
6	CryEngine	$(0 \times 5) + (0 \times 4) + (2 \times 3) + (0 \times 2) + (0 \times 1)$	6

Отримані результати чітко показують, що лідером став Godot Engine. Хоча Unity отримав більше перших місць, Godot Engine переміг завдяки тому, що найвпливовіший експерт (Студент) поставив його на перше місце, тоді як Unity опинився у нього лише на третій позиції. Даний тестовий приклад успішно доводить математичну коректність роботи реалізованого алгоритму.

3.5 Екранні форми додатку

Інтерфейс колективного вибору розроблений з урахуванням вимог до зручності використання та логічно розділений на два блоки. Перший блок містить зведену матрицю думок, яка наочно демонструє розподіл переваг між усіма п'ятьма залученими експертами. Рядками у цій таблиці виступають місця від першого до шостого, а стовпцями є імена експертів із зазначенням їхньої ваги. Таке подання дозволяє аналітику миттєво оцінити загальну картину голосування та знайти точки збігу думок.

 **Колективний вибір**

 **Профіль голосування**
Показано, як експерти розподілили місця між рушіями.

МІСЦЕ	СТУДЕНТ (ФАКТИЧНО ГОЛОВНИЙ РОЗРОБНИК) (ВАГА: 5)	КЕРІВНИК ККП (ВАГА: 4)	ВИКЛАДАЧ УНІВЕРСИТЕТУ (ВАГА: 3)	GAME DESIGNER (ВАГА: 2)	QA ENGINEER (ВАГА: 1)
1	Godot Engine	Unity	Unreal Engine 5	Defold	Unity
2	Unreal Engine 5	Unreal Engine 5	Unity	Godot Engine	Godot Engine
3	Unity	Godot Engine	Godot Engine	GameMaker	Unreal Engine 5
4	Defold	GameMaker	CryEngine	Unity	GameMaker
5	GameMaker	Defold	GameMaker	Unreal Engine 5	Defold
6	CryEngine	CryEngine	Defold	CryEngine	CryEngine

Рисунок 3.1 – Екранна форма профілю голосування експертної групи

Другий блок інтерфейсу відповідає за демонстрацію результатів роботи математичного алгоритму. Таблиця містить не лише кінцевий підсумковий бал, але й детальну розшифровку проміжних розрахунків для кожного ігрового рушія. Користувач бачить точну формулу додавання добутків, що забезпечує повну прозорість процесу прийняття рішення і підвищує рівень довіри до автоматизованої системи. Рядок з рушієм переможцем візуально виділяється серед інших.

Результати розрахунку			
Формула: Сума (Бали_за_місце × Вага_експерта), де Бали = 6 - Місце.			
МІСЦЕ	НАЗВА РУШІЯ	ДЕТАЛЬНИЙ РОЗРАХУНОК	ПІДСУМКОВИЙ БАЛ
1	Godot Engine	$(5 \times 5) + (3 \times 4) + (3 \times 3) + (4 \times 2) + (4 \times 1)$	58
2	Unity	$(3 \times 5) + (5 \times 4) + (4 \times 3) + (2 \times 2) + (5 \times 1)$	56
3	Unreal Engine 5	$(4 \times 5) + (4 \times 4) + (5 \times 3) + (1 \times 2) + (3 \times 1)$	56
4	Defold	$(2 \times 5) + (1 \times 4) + (0 \times 3) + (5 \times 2) + (1 \times 1)$	25
5	GameMaker	$(1 \times 5) + (2 \times 4) + (1 \times 3) + (3 \times 2) + (2 \times 1)$	24
6	CryEngine	$(0 \times 5) + (0 \times 4) + (2 \times 3) + (0 \times 2) + (0 \times 1)$	6

Рисунок 3.2 – Результати математичного розрахунку з урахуванням ваги експертів

3.6 Висновки

У ході виконання третьої лабораторної роботи було успішно запроектовано та програмно реалізовано метод колективного прийняття рішень з урахуванням компетентності експертів. Було здійснено фінальний перехід від наборів розрізнених індивідуальних переваг до єдиного, математично обґрунтованого групового рішення. Таким чином завершено повний цикл роботи системи підтримки прийняття рішень.

Розроблена програмна система автоматично формує зведений профіль голосування на основі збережених у базі даних рангів та виконує швидкий безпомилковий математичний розрахунок за модифікованим правилом Борда. Детальна візуалізація результатів повністю підтвердила ефективність обраного методу.

Ігровий рушій Godot Engine став абсолютним колективним переможцем, набравши п'ятдесят вісім зважених балів. Він випередив своїх найближчих конкурентів Unity та Unreal Engine завдяки стабільно високим оцінкам від експертів з найбільшою вагою у даному проекті. Це доводить, що врахування кваліфікації осіб, які приймають рішення, дозволяє отримати більш якісний та професійно орієнтований результат вибору технологічного забезпечення.

ДОДАТОК А

Відеодемонстрація роботи додатку

Відеозапис з захистом лабораторної роботи доступний за посиланням: <https://youtu.be/QzxJCMJ-aqY>

ДОДАТОК Б

Вихідний код проекту

Вихідний код клієнтської та серверної частин програмного додатку доступний у репозиторії за посиланням: <https://github.com/NurePoturaikoNazar/MOTI-Optimization-Methods-and-Game-Theory>

ДОДАТОК В

Фрагмент серверного коду розрахунку балів

```

router.get('/calculate', async (req, res) => {
  try {
    const rows = await db.allAsync(`
      SELECT
        Alternative.Alternative_id, Alternative.Alternative_name,
        LPR.LPR_name, LPR.LPR_range,
        Result.Alternative_range
      FROM Result
      JOIN LPR ON Result.LPR_id = LPR.LPR_id
      JOIN Alternative ON Result.Alternative_id =
Alternative.Alternative_id
      ORDER BY Alternative.Alternative_id ASC, LPR.LPR_id ASC
    `);

    const altMap = new Map();
    rows.forEach(row => {
      if (!altMap.has(row.Alternative_id)) {
        altMap.set(row.Alternative_id, {
          Alternative_name: row.Alternative_name,
          totalScore: 0,
          calculations: []
        });
      }
      const alt = altMap.get(row.Alternative_id);

      const altPoints = 6 - row.Alternative_range;
      const expertWeight = 6 - row.LPR_range;

      alt.totalScore += altPoints * expertWeight;
      alt.calculations.push(`(${altPoints}×${expertWeight})`);
    });

    const results = Array.from(altMap.values());
    results.sort((a, b) => b.totalScore - a.totalScore);

    res.json(results);
  } catch (err) {
    console.error('Error calculating collective score:', err);
    res.status(500).json({ error: 'Failed to calculate collective
score' });
  }
});

```